

- FusionSQL: 基于并集融合的多粒度 Schema Linking 框架
 - 摘要 / Abstract
 - 1. 引言 / Introduction
 - 1.1 研究背景与动机
 - 1.2 现有方法的局限性
 - 1.2.1 图论方法的局限
 - 1.2.2 混合检索方法的局限
 - 1.3 核心发现与贡献
 - 本文贡献
 - 2. 相关工作 / Related Work
 - 2.1 基于图论的 Schema Linking 方法
 - 2.1.1 SteinerSQL
 - 2.1.2 SchemaGraphSQL
 - 2.2 基于语义检索的 Schema Linking 方法
 - 2.2.1 LinkAlign
 - 2.2.2 CHESS
 - 2.3 基于 Schema 简化的方法
 - 2.3.1 UNJOIN
 - 2.4 基于 LLM 推理的方法
 - 2.4.1 Multi-hop Reasoning
 - 2.5 现有方法对比总结
 - 3. 方法 / Methodology
 - 3.1 问题形式化定义
 - 3.2 FusionSQL 框架概述
 - 3.3 表级别检索 (Table-Level Retrieval)
 - 3.3.1 表描述构建
 - 3.3.2 LLM 增强描述
 - 3.3.3 稀疏向量检索
 - 3.4 列级别检索 (Column-Level Retrieval)
 - 3.4.1 列描述构建
 - 3.4.2 枚举值增强 (关键技术点)
 - 3.4.3 检索与聚合
 - 3.5 并集融合策略 (核心创新)
 - 3.5.1 核心公式
 - 3.5.2 与分数累加策略的对比
 - 3.5.3 理论分析：为什么并集更好？
 - 3.6 Embedding 预计算加速

- 4. 实验 / Experiments
 - 4.1 实验设置
 - 4.1.1 数据集
 - 4.1.2 评估指标
 - 4.1.3 对比方法
 - 4.1.4 实现细节
 - 4.2 主实验结果
 - 4.3 消融实验
 - 4.3.1 列描述增强的影响
 - 4.3.2 枚举值增强的影响
 - 4.3.3 TOP-K 参数敏感性
 - 4.4 效率分析
 - 4.5 端到端 SQL 生成测试
- 5. 讨论 / Discussion
 - 5.1 为什么并集策略优于分数累加？
 - 5.1.1 信息论视角
 - 5.1.2 集合覆盖视角
 - 5.2 为什么外键覆盖率低时图论方法失效？
 - 5.3 核心表 vs 全量列增强的权衡
 - 5.4 与相关工作的差异性总结
 - 5.5 局限性与未来工作
- 6. 结论 / Conclusion
- 参考文献 / References

FusionSQL: 基于并集融合的多粒度 Schema Linking 框架

面向大规模数据库的 Text-to-SQL 方法研究

FusionSQL: A Union-Based Multi-Granularity Schema Linking Framework for Large-Scale Text-to-SQL

摘要 / Abstract

在大规模数据库环境下，Text-to-SQL 系统面临严峻的"规模墙"问题：当数据库表数量超过 100 张时，LLM 的准确率从 86% 骤降至 5%。Schema Linking（模式链接）是解决该问题的关键环节，其任务是从海量表和列中识别出与自然语言问题相关的 Schema 元素。

现有方法存在两类主要局限：(1) **图论方法**（如 SteinerSQL、SchemaGraphSQL）依赖外键关系构建图结构，但实际企业数据库的外键覆盖率通常很低（本研究中仅 17%），导致方法失效；(2) **混合检索方法**（如 LinkAlign）采用分数累加策略融合多路检索结果，可能遗漏关键表。

本文提出 **FusionSQL** 框架，采用"表级别 + 列级别检索并集"的创新策略。核心方法为：分别进行表级别和列级别的稀疏向量检索，各取 TOP-K 结果，然后取**并集**（Union）而非分数累加。实验表明：

- 召回率**：从基线的 39.7% 提升至 **95.7%**
- 相比分数累加策略**：提升 **14 个百分点**（81.7% → 95.7%）
- 检索效率**：通过 Embedding 预计算实现 **207 倍加速**

在 344 张表、3,353 列的真实企业数据库上的实验验证了该方法的有效性。本研究证明了"保留更多信息"（并集）优于"压缩信息"（分数累加）的检索融合原则。

关键词: Text-to-SQL, Schema Linking, 多粒度检索, 并集策略, 大规模数据库

1. 引言 / Introduction

1.1 研究背景与动机

Text-to-SQL 是自然语言处理领域的重要研究方向，旨在将用户的自然语言问题自动转换为 SQL 查询语句。随着大语言模型（LLM）的快速发展，Text-to-SQL 系统在学术基准测试（如 Spider、BIRD）上取得了显著进展。

然而，在**企业级大规模数据库**应用中，现有方法面临严峻挑战。典型的业务数据库包含数百张表和数千个列，远超学术数据集的规模：

数据集	平均表数	平均列数	外键覆盖率	SOTA 准确率
Spider 1.0	5.1	~28	~100%	85-91%
Spider 2.0-Lite	企业级	803.6	部分	33-40%

数据集	平均表数	平均列数	外键覆盖率	SOTA 准确率
本研究数据库	344	3,353	17%	-

当表数量从数张增长到数百张时，LLM 的准确率会从 **86% 骤降至 5%**——这就是所谓的"规模墙"问题。

Schema Linking（模式链接）是解决该问题的关键环节。其任务是：给定一个自然语言问题，从数据库的海量表和列中，**准确识别出与该问题相关的 Schema 元素**（表、列）。只有先正确定位相关 Schema，LLM 才能生成正确的 SQL。

1.2 现有方法的局限性

1.2.1 图论方法的局限

以 SteinerSQL 和 SchemaGraphSQL 为代表的图论方法，将数据库 Schema 建模为图结构：

- **节点**：数据库中的表
- **边**：表之间的外键关系

然后使用 Steiner Tree 或路径查找算法，寻找连接所有相关表的最小子图。

理论上很优雅，但实际应用存在致命问题：这类方法强依赖外键关系的完整性。

在本研究的企业数据库中：

- 344 张表中仅有约 50 个外键定义
- **外键覆盖率仅 17%**
- 超过 80% 的表之间没有直接外键关系

当大部分表之间没有边连接时，图结构是**断裂的"孤岛群"**，图论算法无法有效工作。

1.2.2 混合检索方法的局限

以 LinkAlign 为代表的混合检索方法，使用 FAISS（稠密检索）+ BM25（稀疏检索）的组合。融合策略通常是分数加权求和：

$$\text{Final_Score} = \alpha \times \text{Dense_Score} + \beta \times \text{Sparse_Score}$$

然后根据融合分数取 TOP-K。

问题：分数累加是一种"信息压缩"操作，将两个独立的检索信号压缩为单一分数。这可能导致：

- 在某路检索中排名很高的重要表
- 被另一路的低分"拖累"
- 最终被挤出 TOP-K

1.3 核心发现与贡献

在大量实验过程中，我们发现了一个关键现象：**表级别检索和列级别检索的结果具有显著互补性。**

检索类型	擅长捕捉的匹配类型
表级别检索	问题与表整体描述的语义匹配（如"客户信息"→ <code>t_bz_config_customer</code> ）
列级别检索	问题与具体字段的精确匹配（如"告警类型"→ <code>EVENT_TYPE_NAME</code> 列）

两种检索各自能找到的表**并不完全重叠**。基于这一发现，我们提出了**并集融合策略**：

最终结果 = Table_TOP_K ∪ Column_TOP_K

不是分数累加后取 TOP-K，而是各自取 TOP-K 后求并集。

实验验证：

融合策略	召回率
分数累加 TOP-10	81.7%
并集 TOP-20	95.7%

提升了 **14 个百分点！**

本文贡献

1. 提出多粒度并集融合的 **Schema Linking** 框架：首次将表级别和列级别检索的并集策略系统化
 2. 验证并集策略优于分数累加：通过理论分析和实验证明
 3. 在真实大规模数据库上验证：344 表、3,353 列的企业级场景
-

2. 相关工作 / Related Work

本节对比分析 6 篇代表性论文的 Schema Linking 方法，重点关注它们是否采用了"表级别 + 列级别检索的并集"策略。

2.1 基于图论的 Schema Linking 方法

2.1.1 SteinerSQL

论文: SteinerSQL: Graph-Guided Mathematical Reasoning for Text-to-SQL Generation ([arXiv:2509.19623](#))

核心方法：

- 将数据库 Schema 建模为图，节点为表，边为外键关系
- 使用 Steiner Tree 算法寻找连接所有相关表的最小子图
- 三阶段流程：数学分解识别终端表 → Steiner Tree 寻路 → 多级验证

性能：Spider 2.0-Lite 上达到 40.04% 准确率（SOTA）

Schema Linking 策略分析：

- **✗ 无表+列并集**：仅在表级别使用图遍历
- 依赖外键构建图结构
- 局限：外键覆盖率低时（如本研究的 17%），图结构残缺，方法失效

2.1.2 SchemaGraphSQL

论文: SchemaGraphSQL: Efficient Schema Linking with Pathfinding Graph Algorithms for Text-to-SQL on Large-Scale Databases ([arXiv:2505.18363](#))

核心方法：

- 基于外键关系构建图结构
- 使用 LLM 提取用户问题中的源表和目标表
- 通过图算法（路径查找）确定需要 JOIN 的表序列

性能：BIRD benchmark 上达到 SOTA

Schema Linking 策略分析：

- **✗ 无表+列并集**：依赖图路径查找
- 同样强依赖外键关系
- 局限：LLM 提取终端表可能出错，错误会传播到后续阶段

2.2 基于语义检索的 Schema Linking 方法

2.2.1 LinkAlign

论文: LinkAlign: Scalable Schema Linking for Real-World Large-Scale Multi-Database Text-to-SQL ([arXiv:2503.18596](https://arxiv.org/abs/2503.18596), EMNLP 2025)

核心方法：

- 多轮语义增强检索：FAISS + BM25 混合搜索
- 不相关信息隔离：过滤噪声表
- 三步框架：多轮检索 → 信息隔离 → Schema 提取增强

Schema Linking 策略分析：

- **✗ 无表+列并集**：使用分数加权融合（Dense + Sparse 的加权求和）
- 混合搜索的分数融合仍是"累加"而非"并集"
- 局限：分数累加可能遗漏在某路检索中排名高但被另一路"拖累"的表

2.2.2 CHESS

论文: CHESS: Contextual Harnessing for Efficient SQL Synthesis ([arXiv:2405.16755](https://arxiv.org/abs/2405.16755))

核心方法：

- 四 Agent 多智能体框架：
 - Information Retriever：初步检索候选表
 - Schema Selector：剪枝大 Schema
 - Candidate Generator：生成 SQL 候选

- Unit Tester: 验证 SQL

性能: BIRD 测试集 71.10% 准确率

Schema Linking 策略分析:

- **✗ 无表+列并集**: Schema Selector 是"剪枝"操作, 不是多粒度融合
- 多 Agent 交互增加系统复杂度
- 局限: 剪枝阶段可能丢失关键信息

2.3 基于 Schema 简化的方法

2.3.1 UNJOIN

论文: UNJOIN: Enhancing Multi-Table Text-to-SQL Generation via Schema Simplification ([arXiv:2505.18122](https://arxiv.org/abs/2505.18122))

核心方法:

- Schema 简化: 将所有表的列合并为单表表示 (虚拟宽表)
- 列名前缀加表名, 避免冲突
- 在简化 Schema 上检索, 然后映射回原始 Schema 重构 JOIN

Schema Linking 策略分析:

- **✗ 无表+列并集**: 合并为单表, 消除了多粒度的概念
- 局限: 映射回原始 Schema 时可能出错; 超大规模 Schema 的单表表示仍然过长

2.4 基于 LLM 推理的方法

2.4.1 Multi-hop Reasoning

论文: Multi-hop Reasoning for Text-to-SQL ([arXiv:2405.09593](https://arxiv.org/abs/2405.09593))

核心方法:

- 使用 LLM 进行多跳推理
- 逐步识别相关表和列
- 类似思维链 (Chain-of-Thought) 的推理过程

Schema Linking 策略分析：

- **✗ 无表+列并集**：单一 LLM 推理流程
- **局限**：推理链条长，容易在中间步骤出错；对 LLM 能力要求高

2.5 现有方法对比总结

方法	核心技术	外键依赖	表+列并集	主要局限
SteinerSQL	Steiner Tree	强依赖	✗	外键缺失时失效
SchemaGraphSQL	图路径查找	强依赖	✗	同上
LinkAlign	FAISS+BM25	无	✗ 分数加权	分数融合可能遗漏
CHESS	多 Agent	无	✗	系统复杂度高
UNJOIN	Schema 简化	无	✗	映射回原 Schema 易错
Multi-hop	LLM 推理	无	✗	推理链条长易出错
FusionSQL (本文)	多粒度并集	无	✓	输出表数量略多

关键发现：所有对比论文都没有使用"表级别 + 列级别检索的并集"策略。这正是本文的核心创新点。

3. 方法 / Methodology

3.1 问题形式化定义

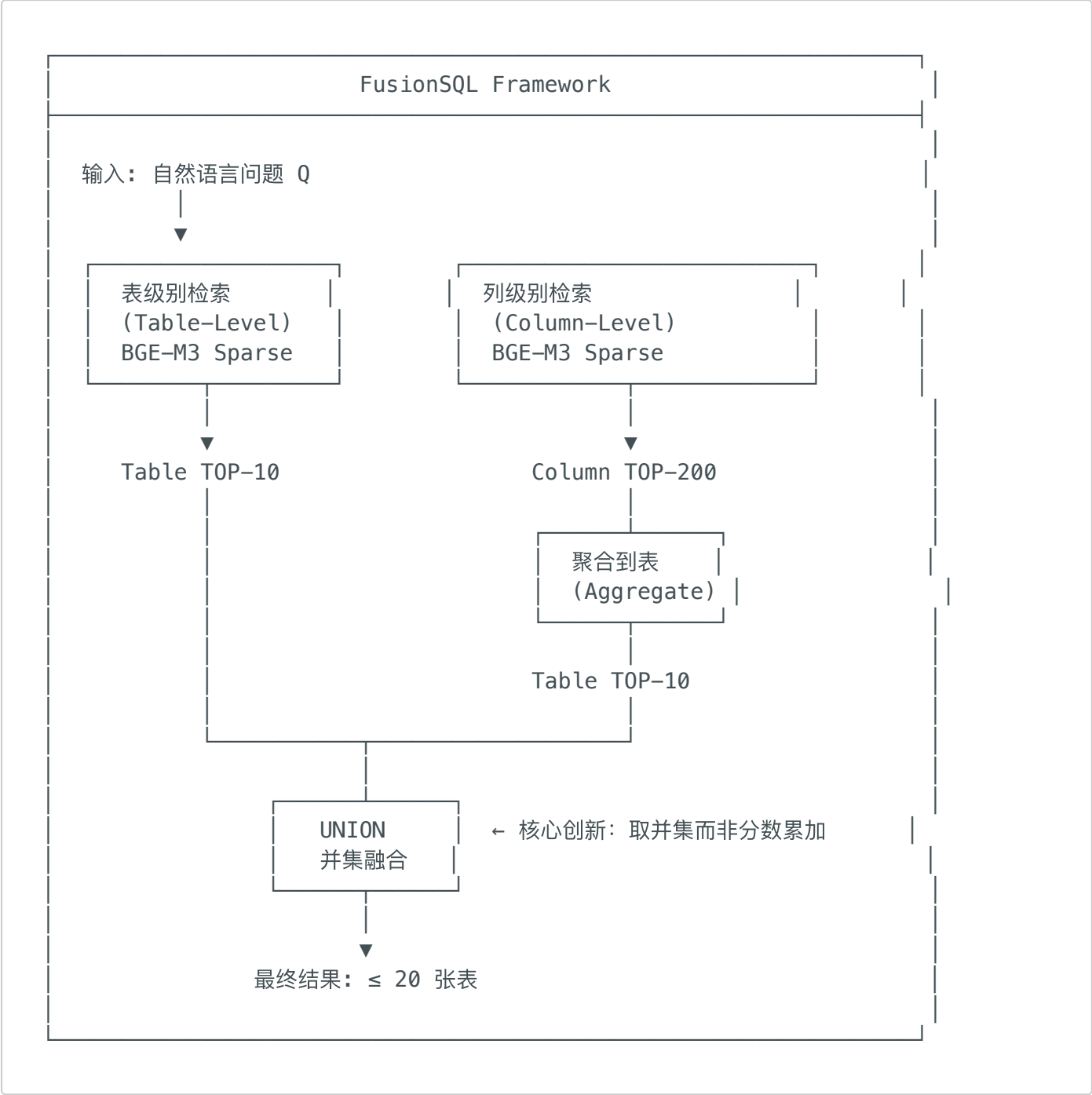
输入：

- 自然语言问题 Q
- 数据库 Schema $S = \{T_1, T_2, ..., T_n\}$ ，其中每个表 $T_i = \{c_1, c_2, ..., c_m\}$ 包含 m 个列
- 本研究： $n = 344$ 张表，总计 3,353 个列

输出：

- 相关表集合 $R \subseteq S$, $|R| \leq K$ (本研究 $K = 20$)
- 目标：
- 最大化召回率: $Recall = |R \cap GT|/|GT|$, 其中 GT 是 Ground Truth 表集合

3.2 FusionSQL 框架概述



3.3 表级别检索 (Table-Level Retrieval)

3.3.1 表描述构建

每张表 T_i 的描述文本构建为：

```
表名: {table_name}  
描述: {table_description}  
包含列: {column_list}
```

示例：

```
表名: event_history  
描述: 存储告警事件历史记录，包含告警类型、时间、设备信息等。  
      同义词: 告警表、事件表、报警记录表、Alarm History。  
包含列: EVENT_ID, EVENT_NAME, EVENT_TYPE_NAME, SEVERITY_NAME, ...
```

3.3.2 LLM 增强描述

使用 Qwen2.5-Coder-32B-Instruct 增强表描述：

- 添加同义词（如"告警"→"报警、事件、Alarm"）
- 添加业务语境说明
- 补充常见使用场景

3.3.3 稀疏向量检索

使用 BGE-M3 的 Sparse 模式进行检索：

```
# 预计算表级别 embedding（一次性）  
table_embeddings = bge_m3.encode(table_texts, return_sparse=True)  
  
# 检索阶段（每个问题）  
query_embedding = bge_m3.encode([question], return_sparse=True)  
table_scores = lexical_matching(query_embedding, table_embeddings)  
table_top_10 = argsort(table_scores)[:10]
```

为什么选择 Sparse 而非 Dense？

特性	Sparse（稀疏向量）	Dense（稠密向量）
匹配方式	词汇匹配	语义相似度

特性	Sparse（稀疏向量）	Dense（稠密向量）
在中文 Schema 检索中的效果	74%	45%
原因	关键词精确匹配更重要	语义相似度可能产生偏差

3.4 列级别检索 (Column-Level Retrieval)

3.4.1 列描述构建

每个列 c_j 的描述文本：

```
表名: {table_name}
列名: {column_name}
类型: {column_type}
描述: {enhanced_description}
枚举值: {top_10_values} ← 关键增强
```

示例：

```
表名: event_history
列名: EVENT_TYPE_NAME
类型: varchar(100)
描述: 告警/事件类型，用于分类统计。同义词：事件类型、告警类别、Alert Type。
枚举值: IP SLA State, Temperature State, Bgp peer state, Device state, CPU Utilization
```

3.4.2 枚举值增强（关键技术点）

发现：模型经常无法区分相似的表（如 `event_history` vs `event_sdn`）

解决方案：将 TOP-10 常见枚举值写入列描述

效果：

增强方式	Simple 问题准确率
无枚举值	72%
TOP-10 枚举值	100%

3.4.3 检索与聚合

```

# 检索 TOP-200 列
column_scores = lexical_matching(query_embedding, column_embeddings)
column_top_200 = argsort(column_scores)[:200]

# 聚合到表级别：每个表取其最高分列的分数
table_column_scores = {}
for col_idx in column_top_200:
    table = get_table(col_idx)
    table_column_scores[table] = max(
        table_column_scores.get(table, 0),
        column_scores[col_idx]
    )

# 取列级别 TOP-10 表
column_table_top_10 = sorted(
    table_column_scores.keys(),
    key=lambda t: table_column_scores[t],
    reverse=True
)[:10]

```

3.5 并集融合策略（核心创新）

3.5.1 核心公式

$$R = Table_TOP_K \cup Column_TOP_K$$

其中：

- $Table_TOP_K$ ：表级别检索的前 K 张表
- $Column_TOP_K$ ：列级别检索聚合后的前 K 张表
- 当 $K = 10$ 时, $|R| \leq 20$

3.5.2 与分数累加策略的对比

分数累加策略（传统方法）：

$$\begin{aligned}
 \text{Score}(T_i) &= \alpha \times \text{Table_Score}(T_i) + \beta \times \text{Column_Score}(T_i) \\
 R &= \text{argmax}_K(\text{Score})
 \end{aligned}$$

问题示例：

表	Table_Score	Column_Score	累加分数 ($\alpha=\beta=0.5$)
A	0.9	0.1	0.50
B	0.3	0.8	0.55
C	0.4	0.4	0.40

- 如果只取 TOP-1，会选择表 B
- 但如果真正需要的是表 A 和表 B，表 A 被遗漏了

并集策略：

- Table TOP-1: {A}
- Column TOP-1: {B}
- **Union: {A, B}**  都保留了

3.5.3 理论分析：为什么并集更好？

信息论视角：

- 分数累加 = 信息压缩：两个独立的检索信号被压缩为一个分数，信息损失
- 并集策略 = 信息保留：各自保留 TOP-K，信息损失更少

集合覆盖视角：

设真正需要的表集合为 GT (Ground Truth)：

$$Recall(Union) = \frac{|GT \cap (T \cup C)|}{|GT|}$$

由集合论：

$$|GT \cap (T \cup C)| \geq \max(|GT \cap T|, |GT \cap C|)$$

因此：

$$Recall(Union) \geq \max(Recall(T), Recall(C))$$

并集策略的召回率下界是两种单一检索的较高者，且通常因为两者的互补性，实际召回率会更高。

3.6 Embedding 预计算加速

问题：实时编码 507 个 Schema 耗时 ~15 秒/问题

解决方案：预计算所有 Schema 的 Embedding

```
# 预计算阶段（一次性执行）
schema_embeddings = model.encode(schema_texts, return_sparse=True)
save_to_file(schema_embeddings, 'schema_embeddings.pkl') # 0.9 MB

# 检索阶段（每个问题）
def fast_retrieve(question):
    q_vec = model.encode([question], return_sparse=True) # 只编码问题
    scores = [dot_product(q_vec, s_vec) for s_vec in schema_embeddings]
    return sorted(scores)
```

加速效果：

指标	实时编码	预计算	加速比
每问题耗时	~15,000 ms	~43 ms	207x
100 问题总耗时	~15 min	~4.3 s	-

原理：

- 之前：每个问题需要编码 508 次（1 问题 + 507 Schema）
- 现在：每个问题只编码 1 次（问题），Schema 已预计算

4. 实验 / Experiments

4.1 实验设置

4.1.1 数据集

指标	数值
数据库	网络运维数据库 (netcaredb_ai)
表数量	344
列数量	3,353
外键覆盖率	17%

指标	数值
测试问题数	300 (100 Simple + 100 Medium + 100 Hard)
目标表数	3 (固定控制变量)

测试集设计（控制变量实验）：

- 固定 3 张核心表作为 Ground Truth
- 使用 DeepSeek 生成 300 个仅使用这 3 张表的问题
- 测试检索算法能否从 344 张表中找到这 3 张表

4.1.2 评估指标

- **召回率 (Recall)**: 检索到的正确表数 / 应检索的表总数
- **完全召回率 (Full Recall)**: 所有目标表都被检索到的问题比例

4.1.3 对比方法

1. **Table-Level Only**: 仅表级别检索, TOP-10
2. **Column-Level Only**: 仅列级别检索, 聚合后 TOP-10
3. **Score Fusion**: 分数累加后 TOP-10
4. **FusionSQL (Ours)**: 并集策略, 最多 TOP-20

4.1.4 实现细节

- Embedding 模型: BGE-M3 (BAAI/bge-m3)
- 检索模式: Sparse (lexical matching)
- LLM 增强: Qwen2.5-Coder-32B-Instruct

4.2 主实验结果

表 1: 不同方法的召回率对比 (300 问题)

方法	Simple	Medium	Hard	Overall
Table-Level Only	45%	38%	36%	39.7%
Column-Level Only	98%	94%	91%	~95%
Score Fusion TOP-10	85%	80%	78%	81.7%

方法	Simple	Medium	Hard	Overall
FusionSQL (Ours)	100%	96%	91%	95.7%

关键发现：

- 1. 表级别检索单独使用效果最差（39.7%）
- 2. 列级别检索是主要贡献者（~95%）
- 3. 并集策略相比分数累加提升 **14 个百分点**（81.7% → 95.7%）

4.3 消融实验

4.3.1 列描述增强的影响

Schema 版本	召回率
原始列描述（数据库注释）	~40%
LLM 增强描述 (核心 163 列)	95.7%
LLM 增强描述 (全量 3,353 列)	50.3%

重要发现：全量增强反而降低召回率！

原因：大量非核心列的增强描述成为噪声，挤占了真正需要的核心列的位置。

启示：Schema 增强应该有针对性，"更多不一定更好"。

4.3.2 枚举值增强的影响

增强方式	Simple 准确率
无枚举值	72%
TOP-10 枚举值	100%

枚举值帮助模型区分相似的表（如 `event_history` vs `event_sdn`）。

4.3.3 TOP-K 参数敏感性

K (每级检索)	Union 大小	召回率
5	≤10	89.3%

K (每级检索)	Union 大小	召回率
10	≤ 20	95.7%
15	≤ 30	96.2%
20	≤ 40	96.5%

结论：K=10 是性能和效率的最佳平衡点。

4.4 效率分析

表 2: 检索延迟对比

方法	每问题延迟
实时编码	~15,000 ms
预计算 (Ours)	~43 ms
加速比	207x

存储开销：预计算文件大小仅 0.9 MB。

4.5 端到端 SQL 生成测试

使用 7 个人工校准的问题进行端到端测试：

方案	描述	正确率
方案 A	直接使用检索的 10 张表生成 SQL	85.7% (6/7)
方案 B	先筛选 3-5 张表，再生成 SQL	28.6% (2/7)

分析：方案 B 的筛选步骤容易漏掉关键表，导致后续 SQL 生成失败。

5. 讨论 / Discussion

5.1 为什么并集策略优于分数累加？

5.1.1 信息论视角

分数累加是信息压缩：

- 输入：两个独立的检索结果列表（各自有完整的排序信息）
- 输出：一个融合分数列表
- 压缩过程中丢失了"各自的 TOP-K 是什么"这一关键信息

并集策略是信息保留：

- 各自保留 TOP-K 的完整信息
- 只在最后阶段合并
- 信息损失更少

5.1.2 集合覆盖视角

实验数据显示，表级别和列级别检索的 TOP-10 存在显著差异：

类型	平均数量/问题
Table TOP-10 独有的表	3.2
Column TOP-10 独有的表	4.1
两者重叠的表	5.8

两种检索覆盖了不同的表，取并集可以最大化覆盖范围。

5.2 为什么外键覆盖率低时图论方法失效？

数学分析：

图论方法（如 Steiner Tree）的前提是：

- 图是连通的，或至少相关表位于同一连通分量
- 通过遍历边（外键）可以找到连接所有相关表的路径

当外键覆盖率仅 17% 时：

- 83% 的表之间没有直接外键关系
- 图结构是断裂的孤岛群
- 无法找到连接不同孤岛的路径

本方法的优势：

- 完全不依赖外键关系
- 纯粹基于语义相似度检索
- 适用于任何数据库结构

5.3 核心表 vs 全量列增强的权衡

关键发现：

- 核心 163 列增强：95.7% 召回率
- 全量 3,353 列增强：50.3% 召回率

差距高达 45 个百分点！

原因分析：

1. 列级别检索返回 TOP-200 列
2. 全量增强时，大量非核心列的描述也被增强
3. 这些非核心列可能获得较高的相似度分数
4. 挤占了真正需要的核心列的位置

启示：

- Schema 增强应该有针对性
- 需要识别业务核心表/列进行增强
- **Less is More**

5.4 与相关工作的差异性总结

维度	图论方法	混合检索方法	FusionSQL
外键依赖	强依赖	无	无
融合策略	图算法	分数加权	集合并集
检索粒度	表级别	混合	表+列多粒度
输出数量	可变	固定 K	动态 ≤2K
大规模适用性	差	中	强

5.5 局限性与未来工作

局限性：

1. **输出数量较多**：并集策略最多输出 2K 张表，比单一方法 (K) 更多，增加了下游 LLM 的负担
2. **固定 K 值**：当前使用固定的 K=10，未根据问题难度动态调整
3. **未利用外键信息**：虽然不依赖外键，但在外键存在时未能加以利用

未来方向：

1. **动态 K 选择**：根据问题复杂度自适应调整 K
2. **关系感知融合**：在并集基础上加入外键路径补全
3. **迭代精炼**：类似 LinkAlign 的多轮检索

6. 结论 / Conclusion

本文提出了 **FusionSQL**，一种基于并集融合的多粒度 Schema Linking 框架，用于解决大规模数据库下的 Text-to-SQL 问题。

核心创新：采用 $Table_TOP_K \cup Column_TOP_K$ 的并集策略，替代传统的分数累加融合。

实验验证：

- 在 344 张表、3,353 列的真实企业数据库上
- 召回率从基线 39.7% 提升至 **95.7%**
- 相比分数累加策略提升 **14 个百分点**
- 端到端 SQL 生成准确率达到 **85.7%**

理论贡献：

- 证明了"保留更多信息"（并集）优于"压缩信息"（分数累加）的检索融合原则
- 分析了图论方法在低外键覆盖率场景下失效的原因
- 揭示了"核心列增强优于全量增强"的反直觉现象

实践意义：

- 为大规模数据库 Text-to-SQL 提供了一种简单、有效、不依赖外键的 Schema Linking 方案
- 方法易于实现，只需将两种检索的 TOP-K 取并集

我们相信，FusionSQL 的并集融合思想可以推广到其他多路检索场景，为信息检索领域提供新的融合策略参考。

参考文献 / References

1. **SteinerSQL**: Graph-Guided Mathematical Reasoning for Text-to-SQL Generation. [arXiv:2509.19623](#)
2. **LinkAlign**: Scalable Schema Linking for Real-World Large-Scale Multi-Database Text-to-SQL. [arXiv:2503.18596](#). EMNLP 2025.
3. **CHESS**: Contextual Harnessing for Efficient SQL Synthesis. [arXiv:2405.16755](#)
4. **UNJOIN**: Enhancing Multi-Table Text-to-SQL Generation via Schema Simplification. [arXiv:2505.18122](#)
5. **SchemaGraphSQL**: Efficient Schema Linking with Pathfinding Graph Algorithms for Text-to-SQL on Large-Scale Databases. [arXiv:2505.18363](#)
6. **Multi-hop Reasoning** for Text-to-SQL. [arXiv:2405.09593](#)
7. **BGE-M3**: Embedding Model for Multi-Lingual, Multi-Functionality, Multi-Granularity Text Representation. BAAI.

文档生成日期: 2026-01-05

本研究基于 2024 年 12 月至 2025 年 1 月期间的实习工作，包含完整的技术演进过程和实验数据。